# Module-Permission-Role and DBSchema

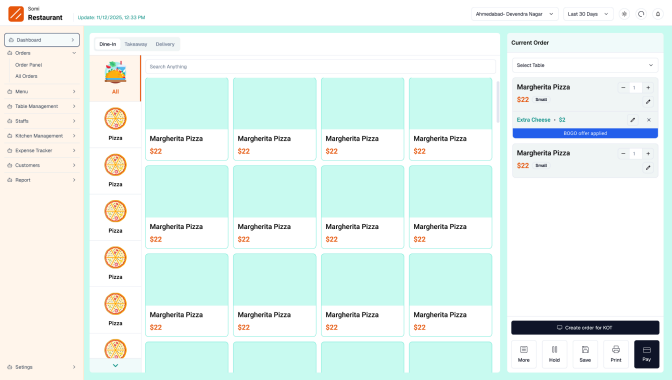
**Module** - broader classification of the whole system into smaller functional units which later can be sold individually. Module is collection of permissions.

**Permissions** - any action or screen requires permission.

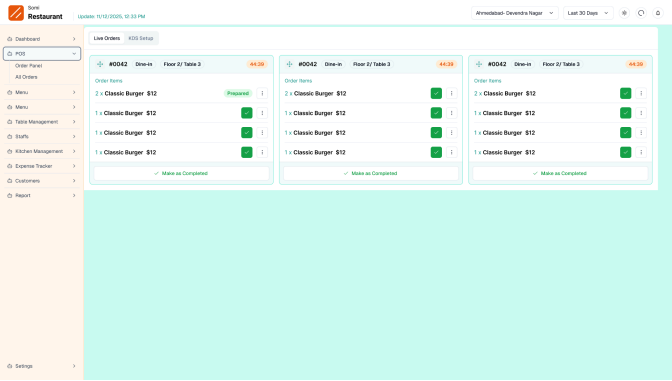
**Roles** - they are fixed defined logical individuals having a fixed set of permissions and thus access to some part/s of module/s.

***** 1 role can have access to multiple permissions of a single module or multiple modules.
* 1 permission can have only 1 module and can be part of roles.

For eg -




**POS Module**
P1 - can_view_menu
P2 - can_place_order
P3 - can_print_slip

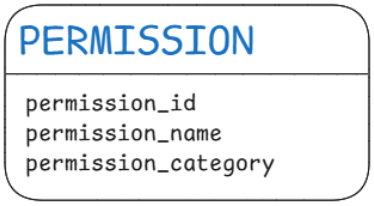



**KDS Module**
P4 - can_view_placed_orders
P5 - can_edit_order_status
…

Now a role ***"Waiter"*** can have multiple permission from multiple modules
Waiter = P1 + P2 + P4
Chef = P1 + P4 + P5
Cashier = P4 + P3

# Database Schema

Permission-role-module related only 2 tables will be made - **permission, user-permission.**

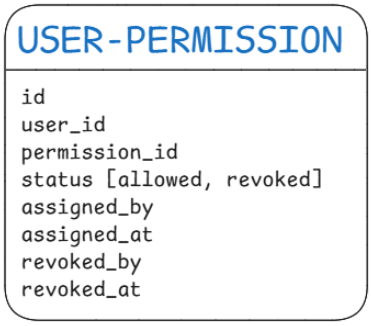



Permission table will have all the set of granular-permission (given as example above) as entries with id, name and category. These are fixed and will not change over time.

**Permission_id**
This will be used in foreign key linking in the user-permission table to map the user to the permission.

**Permission_name**
Names of the permission which will also be hardcoded in the backend and frontend as enum and then will later be used in the both backend and frontend for conditioning based on these permission-names.
Keep in mind the permission name both in the backend and frontend and database should be the same and no duplicacy will be allowed.

**Permission_category**
Permission_category is the module name of the permission.
Later this permission_category will be used to show all the module's permission to the tenant admin and will allow to put all the permission with the specified permission_category to the user.




This is the user-permission table which will map the user with the permission they are assigned. All permission mapping will be there whether the permission is given through a role or directly.

**User_id** - foreign key relation for user in user table
**Permission_id** - foreign key relation for permission table
**Status** - when created a new entry in table this field will be set to allowed. Make an enum for this field either in the backend or in the database. (in the backend would be preferred).
**Assigned_by** - the user's foreign key who have given this role either individually or by role
**Revoked_by** - the user's foreign key who have taken the role back either individually or by the role

| USER |
| --- |
| empolyee_id<br>tenant_id<br>employee_code<br>username<br>name<br>email<br>phone_number<br>password_hash<br>status<br>role<br>current_salary<br>gender<br>joining_date<br>end_date<br>created_at<br>updated_at |

Then in the **user table** you just have to put a field of role. Which is just a role name (a string). This role thing is not mapped to a table nor an enum.

Only 1 role could be assigned to a user

Role in our case will be just useful at the time of giving permission set to user since role has a set of permission so helpful for giving bulk role at once but after that role will not be used at all.

No tenant-specific role-permission mapping has to be done. This mapping is fixed in backend config (we will discuss this later) and will not be the same for all tenants without any change.

## Backend and DataFlow

You have to make configs of permissions and role for following purpose-

- Consistency of name of permission and roles in frontend, backend and database
- Backend's Permission config will be responsible for creation of permission table
- This role config will be used to give the permissions of a user while assigning a role to the user. This role name will be stored in the user table and the permissions in the list will be stored as separate entries in the database.
- **Below is an example of config. This is not exactly how the config should be made (this is just an example).**

```
// permission config
"perm1_name" : "KOT",
"perm2_name" : "KOT",
"perm3_name" : "POS",

// role config
"chef" : ["perm1_name", "perm2_name"],
"waiter" : [],
```

### Authorization

- Role will not be used in authorization

- Each api call will have a set of permission which can call it. Fetch the user's permission set to check if the user has those permissions to run this api

## Dataflow

- Tenant created
- Staff added with specific role in frontend
    - Backend will fetch all permission lists from the config and make individual entries in the permission-user table with foreign key linking with user and permission tables.
    - The backend will store the role in the role field of the user table.
- Staff adds a singular permission to the user
    - Backend will make another entry for this in the permission-user table.

- Staff removes the permission from a user
    - Backend uses the status field in the user-permission table to mark it as revoked permission.
- Staff removes the role from a user
    - Backend sees list of permissions of role from backend
    - Marks all the entries in the user-permission table whose user is told by frontend and whose permission is in the list of permission of role.
    - Remove the role_name from the user table entry and put it null